

Prompting Fundamentals on Real Tasks v2

FL-02 — General AI Fluency, Week 2

Abdul Salam
Backend & AI Systems Engineer

The Real Task

Target task from the FL-01 workflow audit: **coding assignment implementation**. This iteration uses the five named techniques required by this assignment: role assignment, context and motivation, few-shot examples, output structure, and step decomposition — deliberately distinct from the general-purpose “layers” used in the separate Prompt Ladder exercise.

Naive Baseline

"Write backend code"

Output (representative excerpt):

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/api/data', methods=['GET'])
def get_data():
    return jsonify({"message": "Hello, world!"})

if __name__ == '__main__':
    app.run(debug=True)
```

Wrong framework, unrelated to any real task, hedges with a generic explainer instead of doing something specific.

Version 1 — Role assignment

"You are a senior FastAPI backend engineer mentoring a junior developer. Write backend code for creating a new task in a to-do app."

Output (representative excerpt):

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class TaskCreate(BaseModel):
    title: str = Field(..., min_length=1)

tasks: list[dict] = []

@app.post("/tasks", status_code=201)
def create_task(task: TaskCreate) -> dict:
    new_task = {"id": len(tasks) + 1, "title": task.title, "done": False}
    tasks.append(new_task)
    return new_task
```

What changed and why:

Assigning a role produced noticeably more idiomatic code unprompted: type hints, docstrings, and Field-based validation, none of which appeared in the naive baseline. The role changed the quality bar, not just the content.

Version 2 — Context and motivation

"...This is for a FlyRank internship assignment, an in-memory to-do API where tasks already have fields id, title, done. The motivation: this code gets reviewed by hiring managers on GitHub, so it needs to look production-credible, not just work."

Output (representative excerpt):

```
@app.post("/tasks", status_code=201)
def create_task(task: TaskCreate) -> dict:
    if not task.title.strip():
        raise HTTPException(status_code=400, detail="Title is required")
    new_task = {"id": len(tasks) + 1, "title": task.title, "done": False}
    tasks.append(new_task)
    return new_task
```

What changed and why:

Naming the real audience (hiring managers) directly changed behavior: it added 400 validation unprompted, reasoning explicitly that a reviewer would judge missing edge-case handling. Context changed what the model decided was worth doing, not just what it knew.

Version 3 — Few-shot example

"...Example: POST /tasks with {'title':'Buy milk'} should return 201 with {'id':2,'title':'Buy milk','done':false}. Follow this exact response shape."

Output (representative excerpt):

```
# Output matches the response shape exactly:  
{"id": 2, "title": "Buy milk", "done": false}
```

What changed and why:

This is the first version where output shape became guaranteed rather than likely. Earlier versions got close by inference; this one got it exactly right because there was nothing left to infer.

Version 4 — Output structure

"...Return your answer in three labeled sections: *CODE* (just the route function), *TEST* (one curl command), *EDGE CASE* (one line naming what it rejects)."

Output (representative excerpt):

```
CODE:
@app.post("/tasks", status_code=201)
def create_task(task: TaskCreate) -> dict:
    if not task.title.strip():
        raise HTTPException(status_code=400, detail="Title is required")
    new_task = {"id": len(tasks) + 1, "title": task.title, "done": False}
    tasks.append(new_task)
    return new_task

TEST:
curl -X POST http://localhost:8000/tasks -H "Content-Type: application/json" -d '{"title":"Buy milk"}'

EDGE CASE:
Rejects an empty or whitespace-only title with a 400.
```

What changed and why:

Structure made the response scannable and directly pasteable into a README without editing — a real practical win, not just tidiness.

Version 5 — Step decomposition

"...Break your response into steps: Step 1 — define the Pydantic model. Step 2 — write validation logic. Step 3 — write the route handler. Step 4 — write the test command. Do each step separately before combining."

Output (representative excerpt):

```
Step 1: class TaskCreate(BaseModel): title: str
Step 2: if not task.title.strip(): raise HTTPException(400, "Title is required")
Step 3: full route handler combining steps 1-2, plus id generation
Step 4: curl command
```

What changed and why:

Forcing separate reasoning steps surfaced a genuine bug that four prior “correct-looking” versions had silently carried the whole time: the id generation strategy ($\text{len}(\text{tasks}) + 1$) breaks once deletes are introduced, since a deleted task's id could be reused, creating a collision. This is the one real “caught something new” moment across all five versions.

Cross-Model Comparison: Claude vs. ChatGPT

The final prompt (Version 5) was run on both models independently. This is a genuine comparison of two real outputs, not a simulated one.

Role assignment stage: a real, specific difference

Both models responded well to the role prompt, but ChatGPT went noticeably further unprompted: it added a full SQLAlchemy model, a database session dependency, a separate response schema, and a suggested production folder structure, none of which was asked for. Claude's version stayed scoped to exactly what was requested: an in-memory list matching the stated context, no database layer invented.

Honest read: ChatGPT's answer looks more impressive, but arguably follows the prompt worse. The stated context said "in-memory list," and ChatGPT overrode that with its own assumption about what "production-credible" should mean. Claude respected the stated constraint more precisely.

Final combined code: real convergence, one real difference

Both models landed on nearly identical final code: trimmed title, 400 on empty, 201 on success, matching response shape. One real difference: ChatGPT explicitly separates TaskCreate (request) from Task (response) as two distinct schemas; Claude's equivalent didn't make that request/response split as explicit. ChatGPT's separation is arguably the more correct REST practice here.

Neither model caught the id-collision bug that Claude's own step-decomposition run flagged earlier in this same exercise, worth noting since that was the standout catch across the whole ladder, and it didn't reproduce on the second, final-prompt-only run of either model.

Final Reusable Template

You are a senior [STACK] engineer mentoring a junior developer.

Task:

Write backend code for [SPECIFIC ACTION].

Context:

This is for [PROJECT/ASSIGNMENT]. The implementation will be reviewed by [HIRING MANAGERS/TECH LEADS] on GitHub, so the code should look production-credible: clean, readable, idiomatic. Do not over-engineer beyond the stated requirements.

Existing schema:

- [field1]
- [field2]

Example:

Request: POST /tasks {"title": "Buy milk"}

Response (201): {"id": 2, "title": "Buy milk", "done": false}

Think through this in order before combining:

Step 1 - data model. Step 2 - validation. Step 3 - route handler.

Step 4 - test command.

Return exactly these sections: CODE, TEST, EDGE CASE.

This version folds in the one thing the cross-model run revealed: stating “do not over-engineer beyond the stated requirements” explicitly, since that's the exact instruction ChatGPT's role-assignment response would have needed to stay scoped to the actual context given.